

Minecompiler 项目测试报告

1. 测试概述

本测试报告汇总了项目中 `tests/lexer`、`tests/valid`、`tests/invalid` 和 `tests/ir` 四类测试结果，覆盖词法分析、SLR 语法分析、AST 构造、语义分析和 LLVM IR 生成等核心阶段。

测试入口包括：

```
● ● ●
make test
make cmake-test
make demo
```

当前自动化测试已全部通过。若本机存在 `clang`，测试脚本会进一步对生成的 `.ll` 文件执行 `clang -c -x ir`，用于验证 LLVM IR 的类型和语法正确性。

2. 词法分析测试

2.1 测试结果总览

测试文件	测试内容	词法分析结果
<code>tests/lexer/sorted_test.sy</code>	分类测试关键字、标识符、运算符、界符、整数、浮点数和非法字符	符合预期
<code>tests/lexer/mixed.sy</code>	综合测试大小写关键字、注释、表达式、浮点数和错误恢复	符合预期

2.2 分类测试： `sorted_test.sy`

测试命令：

```
● ● ●
./cmmc --lex tests/lexer/sorted_test.sy
```

该用例按类别集中验证所有主要词法单元，包括关键字、标识符、运算符、界符、整数常量、浮点常量和非法字符。

```

1 // KW test
2 INT int InT VoId RetuRn Const MAIn FLOAT if eLse
3
4 // IDN test
5 a _a 4a a4 _4 _a main
6
7 // OP test
8 + - * / = % > < == <= >= != && || <=> === ====
9
10 // SE test
11 ( ) { } ; ,
12
13 // INT & FLOAT test
14 1 100100111120 2.7 31415926.535897
15
16 // ERROR test
17 @
18 &
19 |
20
21 | | | | | ' "

```

关键字测试结果如下。词法分析器能够正确识别所有关键字，并且对大小写不同的关键字执行一致处理，编号与 C-- 词法编号匹配。

INT	<KW,1>
int	<KW,1>
InT	<KW,1>
VoId	<KW,2>
RetuRn	<KW,3>
Const	<KW,4>
MAIn	<KW,5>
FLOAT	<KW,6>
if	<KW,7>
eLse	<KW,8>

标识符测试结果如下。`a`、`_a`、`a4`、`_4_`、`_a_` 等合法标识符均被正确识别；错误形式 `4a` 没有被整体误判为标识符；关键字也不会被错误归类为标识符。

a	<IDN,a>
_a	<IDN,_a>
4	<INT,4>
a	<IDN,a>
a4	<IDN,a4>
4	<IDN,_4_>
a	<IDN,_a_>
main	<KW,5>

运算符测试结果如下。标准运算符均被正确识别并映射到对应编号。对于 `<=>`、`===`、`====` 等非标准多字符符号，词法分析器按照最长匹配原则合理切分，例如 `<=>` 被切分为 `<=` 和 `>`，`===` 被切分为 `==` 和 `=`。

```

+      <OP,6>
-      <OP,7>
*      <OP,8>
/      <OP,9>
=      <OP,11>
%      <OP,10>
>      <OP,12>
<      <OP,13>
==     <OP,14>
<=     <OP,15>
>=     <OP,16>
!=     <OP,17>
&&     <OP,18>
||     <OP,19>
<=     <OP,15>
>      <OP,12>
==     <OP,14>
=       <OP,11>
==     <OP,14>
==     <OP,14>

```

界符测试结果如下。圆括号、花括号、逗号、分号等界符均能被正确识别，并映射到项目内置编号。

```

(      <SE,20>
)      <SE,21>
{      <SE,22>
}      <SE,23>
;      <SE,24>
,      <SE,25>

```

整数和浮点数测试结果如下。整数常量输出为 `<INT,数值>`，浮点数常量输出为 `<FLOAT,数值>`，数值文本保持原样，没有把带小数点的数字错误拆分。

```

1      <INT,1>
100100111120 <INT,100100111120>
2.7    <FLOAT,2.7>
31415926.535897 <FLOAT,31415926.535897>

```

错误符号测试结果如下。`@`、`&`、`|`、`'`、`"` 等语言定义之外的字符均被识别为错误符号，并输出错误类型及行列位置。扫描过程没有因非法字符中断，具备错误恢复能力。

```

@      <ERROR,17,1>
&      <ERROR,18,1>
|      <ERROR,21,17>
'      <ERROR,21,19>
"      <ERROR,21,21>

```

2.3 综合测试：mixed.sy

测试命令：

```

./cmmc --lex tests/lexer/mixed.sy

```

该用例模拟一段正常程序，并额外加入大小写混合关键字、单行注释、浮点数、复合运算符和非法字符，用于验证词法分析器在真实程序场景下的综合能力。

```

1  InT Main() {
2      // keyword case and comments
3      float _v1 = 1.25;
4      int x = 10;
5      if (x >= 10 && _v1 <= 2.0) {
6          return x;
7      }
8      @
9  }

```

```

(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --lex tests\lexer\mixed.sy
InT      <KW,1>
Main     <KW,5>
(        <SE,20>
)        <SE,21>
{        <SE,22>
float    <KW,6>
_v1      <IDN,_v1>
=        <OP,11>
1.25     <FLOAT,1.25>
;        <SE,24>
int      <KW,1>
x        <IDN,x>
=        <OP,11>
10       <INT,10>
;        <SE,24>
if       <KW,7>
(        <SE,20>
x        <IDN,x>
>=       <OP,16>
10       <INT,10>
&&       <OP,18>
_v1      <IDN,_v1>
<=       <OP,15>
2.0      <FLOAT,2.0>
)        <SE,21>
{        <SE,22>
return   <KW,3>
x        <IDN,x>
;        <SE,24>
}        <SE,23>
@        <ERROR,8,3>
}        <SE,23>
lexer:8:3: illegal character '@'
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --lex tests\lexer\extraerrors.s

```

测试结果表明，**InT**、**Main** 能够被识别为关键字，**// keyword case and comments** 被完整忽略，**_v1** 和 **x** 被识别为标识符，**>=**、**<=**、**&&** 等复合运算符按照最长匹配原则整体识别。整数 **10**、浮点数 **1.25** 和 **2.0** 均识别正确。文件末尾故意加入的非法字符 **@** 被输出为错误 token，并给出准确行列位置。

2.4 词法测试结论

词法分析器对合法词法单元的识别完整准确，对非标准多字符符号能够按照最长匹配原则合理拆分，对非法字符能够定位并继续扫描。分类测试与综合测试均符合预期，说明词法分析阶段实现正确且稳定。

3. Valid 正确程序测试

3.1 测试结果总览

测试文件	SLR 语法测试	AST 测试	语义测试
tests/valid/basic.sy	通过	正确	正确
tests/valid/if_else.sy	通过	正确	正确
tests/valid/dangling_else.sy	通过	正确	正确
tests/valid/float_mix.sy	通过	正确	正确
tests/valid/functions.sy	通过	正确	正确
tests/valid/nested_calls.sy	通过	正确	正确

3.2 basic.sy：基础功能测试

测试目标是验证全局变量、局部变量、变量赋值、整数四则运算和 `return` 语句等基础语法与语义。

SLR 分析能够从表达式逐级归约为 `exp`、`stmt`、`block`、`funcDef`、`compUnit`，最终 `accept`。AST 中正确生成 `Program`、`compUnit`、`decl`、`funcDef`、`block` 等节点，结构与源码一致。语义分析中符号表输出正确，没有重定义、未定义使用或类型不匹配。

minecompiler

aca90a72

...

打开的编辑器

basic.sy tests\valid M

basic.txt result U

README.md M

源代码管理: 更改

MINECOMPILER

docs

external

include

ast.h

common.h M

grammar.h

ir.h

lexer.h

semantic.h

token.h

result

basic.txt U

dangling_else.txt U

scripts

run_tests.sh

src

ast.cpp

grammar.cpp

ir.cpp

lexer.cpp

```

90 90 return#return move
91 91 Ident#a move
92 92 lVal#lVal reduction
93 93 primaryExp#lVal reduction
94 94 unaryExp#primaryExp reduction
95 95 mulExp#unaryExp reduction
96 96 addExp#mulExp reduction
97 97 relExp#addExp reduction
98 98 eqExp#relExp reduction
99 99 lAndExp#eqExp reduction
100 100 lOrExp#lAndExp reduction
101 101 exp#lOrExp reduction
102 102 returnExpOpt#exp reduction
103 103 ;; move
104 104 simpleStmt#return reduction
105 105 matchedStmt#simpleStmt reduction
106 106 stmt#matchedStmt reduction
107 107 blockItem#stmt reduction
108 108 blockItemList#blockItemList reduction
109 109 }#} move
110 110 block#{ reduction
111 111 funcDef#bType reduction
112 112 compUnitItem#funcDef reduction
113 113 compUnit#compUnit reduction
114 114 EOF#EOF move
115 115 Program#compUnit reduction
116 116 Program#EOF accept
117

```

```

simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --ast tests/valid/basic.sy
Program @1:1
  compUnit @1:1
    compUnit @1:1
      compUnit @1:1
        compUnitItem @1:1
          decl @1:1
            varDecl @1:1
              bType @1:1
                int: int @1:1
              varDef @1:5
                Ident: a @1:5
                =: = @1:7
                initVal @1:9
                  exp @1:9
                    lOrExp @1:9
                      lAndExp @1:9
                        eqExp @1:9
                          relExp @1:9
                            addExp @1:9
                              mulExp @1:9
                                unaryExp @1:9
                                  primaryExp @1:9
                                    number @1:9
                                      IntConst: 10 @1:9
                                varDefTail @1:1
                                  ;; @1:11
          compUnitItem @3:1
            funcDef @3:1
              bType @3:1
                int: int @3:1
              funcName @3:5
                main: main @3:5
              (: ( @3:9
                funcFParamsOpt @1:1
              ): ) @3:10
              block @3:12

```

```

(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler> ./cmmc --semantic tests/valid/basic.sy
scope  name  kind  type  const  params
global main  function  int    no
global a    variable  int    no
function:main  b    variable  int    no

```

3.3 `if_else.sy`：分支控制流测试

测试目标是验证 `if/else` 分支结构、条件表达式和控制流跳转逻辑。

语法分析能够正常归约嵌套分支结构，AST 中 `if`、`else`、`cond`、`stmt` 等节点层次清晰，语义分析确认函数和局部变量作用域合法，条件表达式类型检查通过。

源代码管理: 存储库

Minecompiler

main*

...

compiler_ir

aca90a72

...

打开的编辑器

basic.sy

tests\valid

M

×

README.md

M

×

if_else.txt

result

U

源代码管理: 更改

MINECOMPILER

include

ast.h

common.h

M

grammar.h

ir.h

lexer.h

semantic.h

token.h

result

basic.txt

U

dangling_else.txt

U

float_mix.txt

U

functions.txt

U

if_else.txt

U

scripts

run_tests.sh

src

ast.cpp

grammar.cpp

result > if_else.txt

117 117 lOrExp#lAndExp reduction

118 118 exp#lOrExp reduction

119 119 returnExpOpt#exp reduction

120 120 ;; move

121 121 simpleStmt#return reduction

122 122 matchedStmt#simpleStmt reduction

123 123 stmt#matchedStmt reduction

124 124 blockItem#stmt reduction

125 125 blockItemList#blockItemList reduction

126 126 }#} move

127 127 block#{ reduction

128 128 simpleStmt#block reduction

129 129 matchedStmt#simpleStmt reduction

130 130 matchedStmt#if reduction

131 131 stmt#matchedStmt reduction

132 132 blockItem#stmt reduction

133 133 blockItemList#blockItemList reduction

134 134 }#} move

135 135 block#{ reduction

136 136 funcDef#bType reduction

137 137 compUnitItem#funcDef reduction

138 138 compUnit#compUnit reduction

139 139 EOF#EOF move

140 140 Program#compUnit reduction

141 141 Program#EOF accept

142

```

simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --ast tests/valid/if_else.sy
Program @1:1
  compUnit @1:1
    compUnit @1:1
      compUnitItem @1:1
        funcDef @1:1
          bType @1:1
            int: int @1:1
          funcName @1:5
            main: main @1:5
          ( : ( @1:9
            funcFParamsOpt @1:1
          ) : ) @1:10
          block @1:12
            { : { @1:12
              blockItemList @1:1
                blockItemList @1:1
                  blockItemList @1:1
                    blockItem @2:3
                      decl @2:3
                        varDecl @2:3
                          bType @2:3
                            int: int @2:3
                          varDef @2:7
                            Ident: a @2:7
                            =: = @2:9
                            initVal @2:11
                              exp @2:11
                                lOrExp @2:11
                                  lAndExp @2:11
                                    eqExp @2:11
                                      relExp @2:11
                                        addExp @2:11
                                          mulExp @2:11
                                            unaryExp @2:11
                                              primaryExp @2:11
                                                number @2:11
                                                  IntConst: 1 @2:11

```

```

(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler> ./cmmc --semantic tests/valid/if_else.sy
scope  name    kind    type    const  params
global main    function int      no
function:main a      variable int      no
function:main b      variable int      no
(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler>

```

3.4 dangling_else.sy: 悬空 else 测试

测试目标是验证经典 dangling else 二义性处理，确保 **else** 与最近的未匹配 **if** 正确绑定。

编译器通过 **matchedStmt / unmatchedStmt** 文法改写消除二义性，SLR 分析路径合法，最终 **accept**。AST 中多层嵌套 **if** 结构归属明确，语义分析中嵌套作用域合法。

源代码管理: 存储库

Minecompiler

main*

...

compiler_ir

aca90a72

...

打开的编辑器

basic.sy

tests\valid

M

dangling_else.txt

result

U

README.md

M

源代码管理: 更改

MINECOMPILER

docs

external

include

ast.h

common.h

M

grammar.h

ir.h

lexer.h

semantic.h

token.h

result

basic.txt

dangling_else.txt

U

scripts

run_tests.sh

src

ast.cpp

grammar.cpp

ir.cpp

lexer.cpp

main.cpp

parser.cpp

大纲

result > dangling_else.txt

99

99

blockItem#stmt

reduction

100

100

blockItemList#blockItemList

reduction

101

101

return#return

move

102

102

IntConst#0

move

103

103

number#IntConst

reduction

104

104

primaryExp#number

reduction

105

105

unaryExp#primaryExp

reduction

106

106

mulExp#unaryExp

reduction

107

107

addExp#mulExp

reduction

108

108

relExp#addExp

reduction

109

109

eqExp#relExp

reduction

110

110

lAndExp#eqExp

reduction

111

111

lOrExp#lAndExp

reduction

112

112

exp#lOrExp

reduction

113

113

returnExpOpt#exp

reduction

114

114

;;

move

115

115

simpleStmt#return

reduction

116

116

matchedStmt#simpleStmt

reduction

117

117

stmt#matchedStmt

reduction

118

118

blockItem#stmt

reduction

119

119

blockItemList#blockItemList

reduction

120

120

}#}

move

121

121

block#{

reduction

122

122

funcDef#bType

reduction

123

123

compUnitItem#funcDef

reduction

124

124

compUnit#compUnit

reduction

125

125

EOF#EOF

move

126

126

Program#compUnit

reduction

127

127

Program#EOF

accept

128

```

simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --ast tests/valid/dangling_else.sy
Program @1:1
  compUnit @1:1
    compUnit @1:1
      compUnitItem @1:1
        funcDef @1:1
          bType @1:1
            int: int @1:1
          funcName @1:5
            main: main @1:5
          (: ( @1:9
            funcFParamsOpt @1:1
          ): ) @1:10
          block @1:12
            { : { @1:12
              blockItemList @1:1
                blockItemList @1:1
                  blockItemList @1:1
                    blockItemList @1:1
                      blockItem @2:3
                        decl @2:3
                          varDecl @2:3
                            bType @2:3
                              int: int @2:3
                            varDef @2:7
                              Ident: a @2:7
                              =: = @2:9
                              initVal @2:11
                                exp @2:11
                                  lOrExp @2:11
                                    lAndExp @2:11
                                      eqExp @2:11
                                        relExp @2:11
                                          addExp @2:11
                                            mulExp @2:11
                                              unaryExp @2:11
                                                primaryExp @2:11
                                                  number @2:11
                                                    IntConst: 0 @2:11
                                                varDefTail @1:1
                                                  ;: ; @2:12
                                                blockItem @3:3

```

```

(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler> ./cmmc --semantic tests/valid/dangling_else.sy
scope  name    kind    type    const  params
global main    function int     no
function:main a      variable int     no

```

3.5 float_mix.sy: 浮点与类型转换测试

测试目标是验证 `float` 类型声明、浮点运算、`int -> float` 隐式转换、浮点函数参数和返回值。

语法分析能够识别 `float` 关键字、浮点常量、浮点声明和函数调用。AST 中包含正确的浮点类型节点、表达式节点和调用节点。语义分析支持 `float` 类型，参数和返回值类型兼容性检查通过。

资源管理器

源代码管理: 存储库

Minecompiler

main*

compiler_ir

aca90a72

打开的编辑器

basic.sy

tests\valid

M

README.md

M

float_mix.txt

result

U

源代码管理: 更改

MINECOMPILER

include

semantic.h

token.h

result

basic.txt

U

dangling_else.txt

U

float_mix.txt

U

scripts

run_tests.sh

src

ast.cpp

grammar.cpp

ir.cpp

lexer.cpp

main.cpp

parser.cpp

semantic.cpp

basic.sy M

README.md M

float_mix.txt U X

result >

float_mix.txt

190 matchedStmt#simpleStmt reduction

191 matchedStmt#if reduction

192 stmt#matchedStmt reduction

193 blockItem#stmt reduction

194 blockItemList#blockItemList reduction

195 }#} move

196 block#{ reduction

197 funcDef#bType reduction

198 compUnitItem#funcDef reduction

199 compUnit#compUnit reduction

200 EOF#EOF move

201 Program#compUnit reduction

202 Program#EOF accept

203

问题

输出

调试控制台

终端

端口

```

simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --ast tests/valid/float_mix.sy
Program @1:1
  compUnit @1:1
    compUnit @1:1
      compUnit @1:1
        compUnitItem @1:1
          funcDef @1:1
            bType @1:1
              float: float @1:1
            funcName @1:7
              Ident: id @1:7
            (: ( @1:9
              funcFParamsOpt @1:10
                funcFParams @1:10
                  funcFParam @1:10
                    bType @1:10
                      float: float @1:10
                    Ident: x @1:16
                  funcFParamsTail @1:1
                ) ) @1:17
            block @1:19
              { : { @1:19
                blockItemList @1:1
                  blockItemList @1:1
                    blockItem @2:3
                      stmt @2:3
                        matchedStmt @2:3
                          simpleStmt @2:3
                            return: return @2:3
                            returnExpOpt @2:10
                              exp @2:10
                                lOrExp @2:10
                                  lAndExp @2:10
                                    eqExp @2:10
                                      relExp @2:10
                                        addExp @2:10
                                          mulExp @2:10
                                            unaryExp @2:10
                                              primaryExp @2:10
                                                lVal @2:10
                                                  Ident: x @2:10
                                                ; : ; @2:11
                                              } : } @3:1

```

```

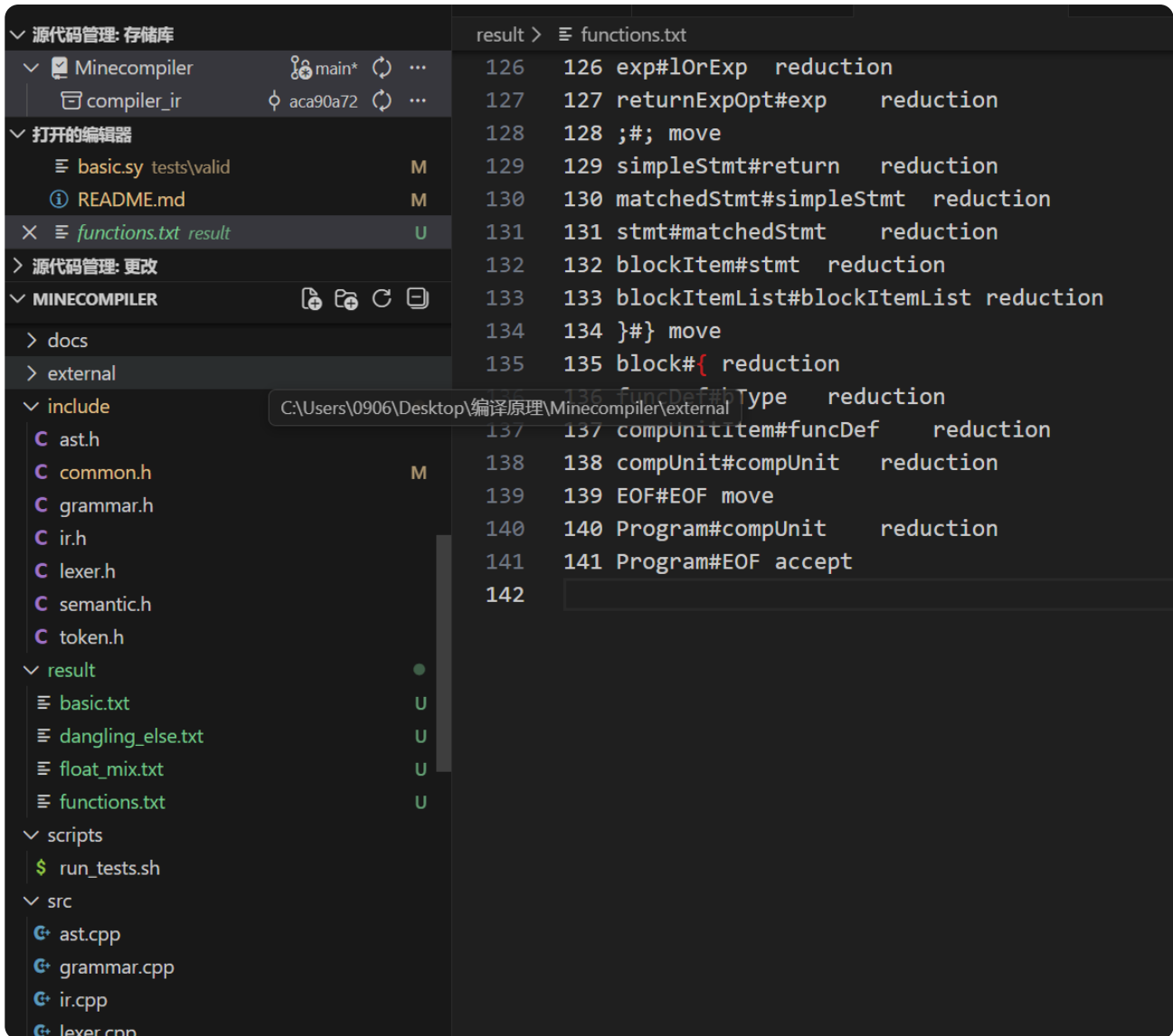
(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler> ./cmmc --semantic tests/valid/float_mix.sy
scope  name    kind    type    const  params
global id      function float   no      float
global main   function int     no
function:id  x      variable float   no
function:main a      variable float   no
function:main b      variable float   no

```

3.6 **functions.sy**：函数定义、调用与参数测试

测试目标是验证多参数函数定义、函数调用、实参与形参匹配、参数传递和返回值传递。

语法分析能够正确归约 **funcFParam**、**funcDef** 和 call 表达式。AST 中函数定义、参数列表、函数调用和返回语句结构完整。语义分析能够记录函数参数列表，并检查函数重复定义、参数个数、参数类型和作用域隔离。



```

● simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --ast tests/valid/functions.sy
Program @1:1
  compUnit @1:1
    compUnit @1:1
      compUnit @1:1
        compUnitItem @1:1
          funcDef @1:1
            bType @1:1
              int: int @1:1
            funcName @1:5
              Ident: add @1:5
            (: ( @1:8
              funcFParamsOpt @1:9
                funcFParams @1:9
                  funcFParam @1:9
                    bType @1:9
                      int: int @1:9
                    Ident: a @1:13
                  funcFParamsTail @1:14
                    ,: , @1:14
                  funcFParam @1:16
                    bType @1:16
                      int: int @1:16
                    Ident: b @1:20
                  funcFParamsTail @1:1
                ) ) @1:21
              block @1:23
                { : { @1:23
                  blockItemList @1:1
                    blockItemList @1:1
                      blockItem @2:3
                        stmt @2:3
                          matchedStmt @2:3
                            simpleStmt @2:3
                              return: return @2:3
                              returnExpOpt @2:10
                                exp @2:10
                                  lOrExp @2:10
                                    lAndExp @2:10
                                      eqExp @2:10
                                        relExp @2:10
                                          addExp @2:10
                                            addExp @2:10
                                              mulExp @2:10
                                                unaryExp @2:10
                                                  primaryExp @2:10
                                                    lVal @2:10

```

```

(base) PS C:\Users\0906\Desktop\编译原理\Minecompiler> ./cmmc --semantic tests/valid/functions.sy
scope  name  kind  type  const  params
global add   function  int    no     int,int
global main  function  int    no
function:add a         variable int    no
function:add b         variable int    no
function:main x         variable int    no

```

3.7 `nested_calls.sy`: 函数嵌套调用测试

测试目标是验证多层嵌套函数调用、函数返回值作为实参传递、无参函数调用、跨函数作用域隔离和参数类型检查。

`f(f(1))` 能够按照 `unaryExp -> exp -> returnExpOpt` 逐级归约，所有表达式、语句和函数定义均正常通过 SLR 分析。AST 能够表示多层函数调用关系。语义分析正确识别全局函数 `f`、`g`、`main` 以及 `f` 的局部参数 `x`，不会把内层调用参数错误计入外层调用。


```
simpleedu@LAPTOP-B57J4MGK:~/Minecompiler$ ./cmmc --semantic tests/valid/nested_calls.sy
scope  name  kind  type  const  params
global f      function  int   no     int
global g      function  int   no
global main   function  int   no
function:f    x        variable int    no
```

3.8 Valid 测试结论

`tests/valid` 中所有正确程序均通过 SLR 语法分析、AST 构造和语义检查。测试覆盖基础声明与表达式、分支控制流、dangling else、浮点扩展、函数定义调用和嵌套调用，说明前端主流程在合法程序上表现稳定。

4. Invalid 错误程序测试

4.1 测试结果总览

测试文件	错误类型	语法分析结果	语义分析结果
<code>tests/invalid/syntax.sy</code>	语法错误	失败，成功报错	未进入语义阶段
<code>tests/invalid/call_args.sy</code>	函数实参数量错误	通过	失败，成功报错
<code>tests/invalid/const_assign.sy</code>	const 常量赋值错误	通过	失败，成功报错
<code>tests/invalid/semantic.sy</code>	未定义变量错误	通过	失败，成功报错
<code>tests/invalid/type_return.sy</code>	返回值类型错误	通过	失败，成功报错

4.2 `syntax.sy`：语法错误测试

测试命令：


```
./cmmc --parse tests/invalid/syntax.sy
```

测试结果为：

```
parser:1:11: unexpected token '{' in state 22
```

该样例中函数定义写成 `int main( {`，在 `main(` 后缺少右括号 `)`，导致分析器在第 1 行第 11 列读取 `{` 时无法继续。编译器能够在语法阶段准确报错，并给出错误位置。

```
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --parse tests/invalid/syntax.sy
1  state0#int      reduction(2)
2  compUnit#int    move
3  int#main        reduction(11)
4  bType#main      move
5  main#(          reduction(24)
6  funcName#(      move
7  ({             error
parser:1:11: unexpected token '{' in state 22
```

4.3 `call_args.sy`：函数参数数量错误测试

测试命令：

```
./cmmc --parse tests/invalid/call_args.sy
./cmmc --semantic tests/invalid/call_args.sy
```

语法分析通过，语义分析输出：

```
semantic:6:10: function 'add' expects 2 argument(s), got 1
```

该样例定义 `int add(int a, int b)`，但在 `main` 中调用 `add(1)`，实参数量不足。该程序语法结构正确，因此 `parse` 阶段通过；语义阶段能够检查函数参数个数并输出明确诊断。

```

(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --parse tests/invalid/call_args.sy
85  addExp#;      reduction(76)
86  relExp#;      reduction(81)
87  eqExp#; reduction(84)
88  lAndExp#;     reduction(86)
89  lOrExp#;      reduction(49)
90  exp#;   reduction(48)
91  returnExpOpt#; move
92  ;#}      reduction(46)
93  simpleStmt#} reduction(38)
94  matchedStmt#} reduction(36)
95  stmt#} reduction(35)
96  blockItem#} reduction(33)
97  blockItemList#} move
98  }#EOF reduction(31)
99  block#EOF reduction(21)
100 funcDef#EOF reduction(5)
101 compUnitItem#EOF reduction(3)
102 compUnit#EOF move
103 EOF#EOF reduction(1)
104 Program#EOF accept
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --semantic tests/invalid/call_args.sy
scope name kind type const params
global add function int no int,int
global main function int no
function:add a variable int no
function:add b variable int no
semantic:6:10: function 'add' expects 2 argument(s), got 1

```

4.4 `const_assign.sy`: `const` 常量赋值错误测试

测试命令:

```

./cmmc --parse tests/invalid/const_assign.sy
./cmmc --semantic tests/invalid/const_assign.sy

```

语法分析通过, 语义分析输出:

```

semantic:4:3: cannot assign to const variable 'a'

```

该样例先声明 `const int a = 1;`, 随后在 `main` 中执行 `a = 2;`。赋值语句语法合法, 但违反 `const` 变量不可再次赋值的语义规则。

```

(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --parse tests/invalid/const_assign.sy
57  mulExp#;      reduction(73)
58  addExp#;      reduction(76)
59  relExp#;      reduction(81)
60  eqExp#; reduction(84)
61  lAndExp#;     reduction(86)
62  lOrExp#;      reduction(49)
63  exp#;   reduction(48)
64  returnExpOpt#; move
65  ;#}      reduction(46)
66  simpleStmt#} reduction(38)
67  matchedStmt#} reduction(36)
68  stmt#} reduction(35)
69  blockItem#} reduction(33)
70  blockItemList#} move
71  }#EOF reduction(31)
72  block#EOF reduction(21)
73  funcDef#EOF reduction(5)
74  compUnitItem#EOF reduction(3)
75  compUnit#EOF move
76  EOF#EOF reduction(1)
77  Program#EOF accept
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --semantic tests/invalid/const_assign.sy
scope name kind type const params
global main function int no
global a variable int yes
semantic:4:3: cannot assign to const variable 'a'

```

4.5 `semantic.sy`：未定义变量错误测试

测试命令：

```
./cmmc --parse tests/invalid/semantic.sy
./cmmc --semantic tests/invalid/semantic.sy
```

语法分析通过，语义分析输出：

```
semantic:5:3: assignment to undefined variable 'b'
semantic:7:10: use of undefined variable 'b'
```

该样例在 `main` 函数中使用了未声明变量 `b`。语义分析器能够在赋值语句和返回语句中分别检测到未定义变量，说明符号表查询和多错误诊断逻辑有效。

```
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --parse tests/invalid/semantic.sy
108  addExp#;      reduction(76)
109  relExp#;      reduction(81)
110  eqExp#; reduction(84)
111  lAndExp#;     reduction(86)
112  lOrExp#;      reduction(49)
113  exp#; reduction(48)
114  returnExpOpt#; move
115  ;#} reduction(46)
116  simpleStmt#} reduction(38)
117  matchedStmt#} reduction(36)
118  stmt#} reduction(35)
119  blockItem#} reduction(33)
120  blockItemList#} move
121  }#EOF reduction(31)
122  block#EOF reduction(21)
123  funcDef#EOF reduction(5)
124  compUnitItem#EOF reduction(3)
125  compUnit#EOF move
126  EOF#EOF reduction(1)
127  Program#EOF accept
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --semantic tests/invalid/semantic.sy
scope name kind type const params
global main function int no
global a variable int yes
function:main a variable int no
semantic:5:3: assignment to undefined variable 'b'
semantic:7:10: use of undefined variable 'b'
```

4.6 `type_return.sy`：返回值类型错误测试

测试命令：

```
./cmmc --parse tests/invalid/type_return.sy
./cmmc --semantic tests/invalid/type_return.sy
```

语法分析通过，语义分析输出：

```
semantic:3:3: return type float does not match int
```

该样例中 `main` 函数返回类型为 `int`，但函数体内返回 `float` 类型变量。语义分析器能够识别返回表达式类型与函数声明返回类型不一致。

```
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --parse tests/invalid/type_return.sy
39 unaryExp#; reduction(69)
40 mulExp#; reduction(73)
41 addExp#; reduction(76)
42 relExp#; reduction(81)
43 eqExp#; reduction(84)
44 lAndExp#; reduction(86)
45 lOrExp#; reduction(49)
46 exp#; reduction(48)
47 returnExpOpt#; move
48 ;#} reduction(46)
49 simpleStmt#} reduction(38)
50 matchedStmt#} reduction(36)
51 stmt#} reduction(35)
52 blockItem#} reduction(33)
53 blockItemList#} move
54 }#EOF reduction(31)
55 block#EOF reduction(21)
56 funcDef#EOF reduction(5)
57 compUnitItem#EOF reduction(3)
58 compUnit#EOF move
59 EOF#EOF reduction(1)
60 Program#EOF accept
(wangzx) wangzixuan@Ruby:~/Compilers/final_lab/Minecompiler$ ./cmmc --semantic tests/invalid/type_return.sy
scope name kind type const params
global main function int no
function:main a variable float no
semantic:3:3: return type float does not match int
```

4.7 Invalid 测试结论

错误程序测试表明，编译器能够区分语法错误和语义错误。语法错误会在 SLR 分析阶段被拦截；语法正确但语义非法的程序能够进入语义阶段，并准确报告函数参数数量错误、const 赋值错误、未定义变量和返回类型错误。

5. IR 生成测试

5.1 测试结果总览

测试文件	测试内容	测试结果
tests/ir/const_expr.sy	全局整数常量表达式	通过
tests/ir/consts.sy	全局整数常量	通过
tests/ir/float_call_cast.sy	浮点函数定义与调用、类型转换	通过
tests/ir/float_const_expr.sy	浮点常量表达式	通过
tests/ir/float_global.sy	浮点全局变量	通过
tests/ir/float_init_expr.sy	浮点常量初始化	通过
tests/ir/float_local_const.sy	局部浮点常量	通过
tests/ir/functions.sy	整数函数定义与调用	通过
tests/ir/if_else.sy	条件分支	通过
tests/ir/simple.sy	基本运算	通过

5.2 `const_expr.sy`：全局整数常量表达式

测试命令：

```
./cmmc --ir tests/ir/const_expr.sy
```

IR 生成器正确处理全局常量定义，并在编译期对 `a + 1` 进行常量折叠，生成 `@b = constant i32 4`。`main` 函数加载 `@b` 并返回，实际输出与预期一致。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir tests\ir\const_expr.sy
; ModuleID = 'sysy2022_compiler'
source_filename = "tests\ir\const_expr.sy"

@a = constant i32 3
@b = constant i32 4
declare i32 @getint()

declare i32 @getch()

declare i32 @getarray(i32*)

declare void @putint(i32)

declare void @putch(i32)

declare void @putarray(i32, i32*)

declare void @starttime()

declare void @stoptime()

define i32 @main() {
label_main_ENTRY:
    %op0 = load i32, i32* @b
    ret i32 %op0
}
```

5.3 `consts.sy`：全局整数常量

测试命令：

```
./cmmc --ir tests/ir/consts.sy
```

该用例包含全局常量 `base = 7` 和局部变量 `delta = 5`。生成 IR 中，`@base` 正确定义为 `constant i32 7`，局部变量通过 `alloca` 分配空间并初始化，随后执行整数加法并返回。

```

(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir tests\ir\consts.sy
; ModuleID = 'sysy2022_compiler'
source_filename = "tests\ir\consts.sy"

@base = constant i32 7
declare i32 @getint()

declare i32 @getch()

declare i32 @getarray(i32*)

declare void @putint(i32)

declare void @putch(i32)

declare void @putarray(i32, i32*)

declare void @starttime()

declare void @stoptime()

define i32 @main() {
label_main_ENTRY:
    %delta.0 = alloca i32
    store i32 5, i32* %delta.0
    %x.1 = alloca i32
    %op0 = load i32, i32* @base
    %op1 = load i32, i32* %delta.0
    %op2 = add i32 %op0, %op1
    store i32 %op2, i32* %x.1
    %op3 = load i32, i32* %x.1
    ret i32 %op3
}

```

5.4 float_call_cast.sy：浮点函数定义与调用

测试命令：

```

./cmmc --ir tests/ir/float_call_cast.sy

```

该用例定义 `float f(float x)`，并在 `g` 中调用 `f(1)`。生成 IR 在调用点插入 `sitofp i32 1 to float`，再将转换后的值作为 `float` 参数传递给 `@f`，验证了 `int -> float` 隐式类型提升。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\float_call_cast.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\float_call_cast.sy"

declare i32 @getint()
declare i32 @getch()
declare i32 @getarray(i32*)
declare void @putint(i32)
declare void @putch(i32)
declare void @putarray(i32, i32*)
declare void @starttime()
declare void @stoptime()

define float @f(float %x.arg) {
f_ENTRY:
  %x = alloca float
  store float %x.arg, float* %x
  %op0 = load float, float* %x
  ret float %op0
}

define float @g() {
g_ENTRY:
  %op2 = sitofp i32 1 to float
  %op1 = call float @f(float %op2)
  ret float %op1
}
```

5.5 float_const_expr.sy：浮点常量表达式

测试命令：

```
./cmmc --ir tests/ir/float_const_expr.sy
```

该用例定义全局常量 `a = 3`、`b = a + 1`，函数 `f` 返回 `b + 0.5`。生成 IR 中，`b` 被折叠为 `4`，函数体内加载整数 `b` 后通过 `sitofp` 转换为浮点，再执行 `fadd`。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\float_const_expr.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\float_const_expr.sy"

declare i32 @getint()
declare i32 @getch()
declare i32 @getarray(i32*)
declare void @putint(i32)
declare void @putch(i32)
declare void @putarray(i32, i32*)
declare void @starttime()
declare void @stoptime()

@a = constant i32 3
@b = constant i32 4

define float @f() {
f_ENTRY:
  %op0 = load i32, i32* @b
  %op1 = sitofp i32 %op0 to float
  %op2 = fadd float %op1, 0.5
  ret float %op2
}
```

5.6 float_global.sy：浮点全局变量

测试命令：

```
./cmmc --ir tests/ir/float_global.sy
```

该用例定义全局浮点变量 `g = 1.5`，`main` 返回 `g > 0.0` 的比较结果。生成 IR 中，`@g` 被正确初始化为浮点全局变量，函数体内加载 `@g`，执行 `fcmp ogt` 比较，并通过 `zext` 扩展为 `i32` 返回。

```
(base) PS D:\CODE\C++_files\Winecompiler-main> .\cmmc.exe --ir .\tests\ir\float_global.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\float_global.sy"

declare i32 @getint()
declare i32 @getch()
declare i32 @getarray(i32*)
declare void @putint(i32)
declare void @putch(i32)
declare void @putarray(i32, i32*)
declare void @starttime()
declare void @stoptime()

@g = global float 1.500000e+00

define i32 @main() {
main_ENTRY:
    %op0 = load float, float* @g
    %op1 = fcmp ogt float %op0, 0.0
    %op2 = zext i1 %op1 to i32
    ret i32 %op2
}
```

5.7 float_init_expr.sy：浮点常量初始化

测试命令：

```
./cmmc --ir tests/ir/float_init_expr.sy
```

该用例包含整数常量 `a = 3`、浮点常量 `h = a + 0.5` 和浮点全局变量 `g = 1`。生成 IR 中，`@h = constant float 3.500000e+00`，`@g = global float 1.000000e+00`，说明全局 float 初始化和常量表达式求值均正确。

```
(base) PS D:\CODE\C++_files\Winecompiler-main> .\cmmc.exe --ir .\tests\ir\float_init_expr.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\float_init_expr.sy"

declare i32 @getint()
declare i32 @getch()
declare i32 @getarray(i32*)
declare void @putint(i32)
declare void @putch(i32)
declare void @putarray(i32, i32*)
declare void @starttime()
declare void @stoptime()

@a = constant i32 3
@h = constant float 3.500000e+00
@g = global float 1.000000e+00

define i32 @main() {
main_ENTRY:
    %op0 = load float, float* @h
    %op1 = load float, float* @g
    %op2 = fcmp ogt float %op0, %op1
    %op3 = zext i1 %op2 to i32
    ret i32 %op3
}
```


5.8 float_local_const.sy: 局部浮点常量

测试命令:

```
./cmmc --ir tests/ir/float_local_const.sy
```

该用例在函数内声明局部常量 `c = 2`，返回 `c + 1.0`。生成 IR 中，局部常量通过 `alloca i32` 分配并初始化，加载后通过 `sitofp` 转换为浮点，再与 `1.0` 执行 `fadd`。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\float_local_const.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\float_local_const.sy"

declare i32 @getint()
declare i32 @getch()
declare i32 @getarray(i32*)
declare void @putint(i32)
declare void @putch(i32)
declare void @putarray(i32, i32*)
declare void @starttime()
declare void @stoptime()

define float @f() {
f_ENTRY:
    %c.0 = alloca i32
    store i32 2, i32* %c.0
    %op1 = load i32, i32* %c.0
    %op2 = sitofp i32 %op1 to float
    %op3 = fadd float %op2, 1.0
    ret float %op3
}
```

5.9 functions.sy: 整数函数定义与调用

测试命令:

```
./cmmc --ir tests/ir/functions.sy
```

该用例定义 `add(i32 a, i32 b)` 并在 `main` 中调用。生成 IR 中，函数参数通过 `alloca` 存储到局部变量，执行加法后返回；`main` 生成 `call i32 @add(i32 10, i32 20)` 指令，参数传递和返回值处理正确。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\functions.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\functions.sy"

declare i32 @getint()

declare i32 @getch()

declare i32 @getarray(i32*)

declare void @putint(i32)

declare void @putch(i32)

declare void @putarray(i32, i32*)

declare void @starttime()

declare void @stoptime()

define i32 @add(i32 %a.arg, i32 %b.arg) {
label_add_ENTRY:
    %a.0 = alloca i32
    store i32 %a.arg, i32* %a.0
    %b.1 = alloca i32
    store i32 %b.arg, i32* %b.1
    %op0 = load i32, i32* %a.0
    %op1 = load i32, i32* %b.1
    %op2 = add i32 %op0, %op1
    ret i32 %op2
}

define i32 @main() {
label_main_ENTRY:
    %x.0 = alloca i32
    %op0 = call i32 @add(i32 10, i32 20)
    store i32 %op0, i32* %x.0
    %op1 = load i32, i32* %x.0
    ret i32 %op1
}
```

5.10 if_else.sy：条件分支

测试命令：

```
./cmmc --ir tests/ir/if_else.sy
```

该用例在 `main` 中定义局部变量 `a = 3`，若 `a > 2` 返回 `1`，否则返回 `0`。生成 IR 中包含比较指令、条件跳转和 then/else 基本块，控制流结构符合源程序语义。

```

(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\if_else.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\if_else.sy"

declare i32 @getint()

declare i32 @getch()

declare i32 @getarray(i32*)

declare void @putint(i32)

declare void @putch(i32)

declare void @putarray(i32, i32*)

declare void @starttime()

declare void @stoptime()

define i32 @main() {
label_main_ENTRY:
  %a.0 = alloca i32
  store i32 3, i32* %a.0
  %op0 = load i32, i32* %a.0
  %op1 = icmp sgt i32 %op0, 2
  %op2 = zext i1 %op1 to i32
  %op3 = icmp ne i32 %op2, 0
  br i1 %op3, label %label_if_then0, label %label_if_else1

label_if_then0:                                ; preds = %label_main_ENTRY
  ret i32 1

label_if_else1:                                ; preds = %label_main_ENTRY
  ret i32 0
}

```

5.11 `simple.sy`：基本运算

测试命令：

```

● ● ●
./cmmc --ir tests/ir/simple.sy

```

该用例定义全局变量 `a`，在 `main` 中计算 `x * y + 2` 并写回 `a`。生成 IR 中包含全局变量读写、局部变量分配、乘法和加法指令，最终加载 `a` 并返回。

```
(base) PS D:\CODE\C++_files\Minecompiler-main> .\cmmc.exe --ir .\tests\ir\simple.sy
; ModuleID = 'sysy2022_compiler'
source_filename = ".\tests\ir\simple.sy"

@a = global i32 0
declare i32 @getint()

declare i32 @getch()

declare i32 @getarray(i32*)

declare void @putint(i32)

declare void @putch(i32)

declare void @putarray(i32, i32*)

declare void @starttime()

declare void @stoptime()

define i32 @main() {
label_main_ENTRY:
    %x.0 = alloca i32
    store i32 3, i32* %x.0
    %y.1 = alloca i32
    store i32 4, i32* %y.1
    %op0 = load i32, i32* %x.0
    %op1 = load i32, i32* %y.1
    %op2 = mul i32 %op0, %op1
    %op3 = add i32 %op2, 2
    store i32 %op3, i32* @a
    %op4 = load i32, i32* @a
    ret i32 %op4
}
```

5.12 IR 测试结论

IR 测试覆盖全局常量表达式、局部变量和全局变量、函数调用、条件分支、基本算术运算、浮点全局变量、浮点常量初始化、局部 const 和 `int -> float` 类型转换。所有用例均输出符合预期的 LLVM IR，并通过自动化测试脚本校验。

6. 总体结论

本次测试覆盖了编译器前端从源程序输入到 LLVM IR 输出的完整链路：

1. 词法阶段能够识别合法 token，支持关键字大小写不敏感、注释过滤、最长匹配和非法字符恢复。
2. 语法阶段能够自动构造 SLR 分析过程，对正确程序输出 `accept`，对错误程序输出带位置的 parser 诊断。
3. AST 构造能够保持程序结构层次，支持声明、函数、表达式、分支和调用节点。
4. 语义阶段能够检查作用域、未定义变量、const 赋值、函数参数、返回值类型和嵌套调用。
5. IR 阶段能够生成全局变量、局部变量、函数调用、条件分支、整型表达式和浮点扩展相关 LLVM IR。

综上，Minecompiler 当前测试用例均已通过，项目实现满足 C-- 编译器前端对词法分析、语法分析、语义分析和中间代码生成的核心要求，并具备较好的错误诊断和回归测试能力。